



SynchroDev
Owen AURIAULT

Documentation SynchroDrop

1. Présentation du projet

Nom : SynchroDrop

Plateforme : Android

Objectif : Créer une messagerie locale sécurisée utilisant le Bluetooth via l'API Google Nearby pour permettre à deux utilisateurs de se connecter et de discuter avec des messages chiffrés.

Fonctionnalités principales :

- Découverte des utilisateurs à proximité via Bluetooth (**Advertising** et **Discovery**).
 - Connexion sécurisée entre deux appareils.
 - Chiffrement et déchiffrement AES des messages.
 - Interface de chat avec affichage différencié des messages envoyés et reçus.
 - Gestion des autorisations Bluetooth et localisation.
-

2. Architecture et choix techniques

2.1 Stack technique

- **Langage** : Java (Android)
- **API principale** : Google Nearby Connections API
- **UI** : Android RecyclerView pour la liste des utilisateurs et la discussion.
- **Chiffrement** : AES (Advanced Encryption Standard) via **AESUtils**.
- **Permissions Android** : BLUETOOTH, ACCESS_WIFI_STATE, ACCESS_COARSE_LOCATION.

2.2 Organisation du projet

- **MainActivity.java** : Gestion de l'activité principale, découverte, publicité, envoi et réception des messages.

- `UserNearby.java` : Modèle pour stocker les informations d'un utilisateur à proximité.
- `MessageNearby.java` : Modèle pour stocker un message envoyé ou reçu.
- `NearbyAdapter.java` : Adapter RecyclerView pour afficher la liste des utilisateurs.
- `ChatAdapter.java` : Adapter RecyclerView pour afficher le chat.
- `AESUtils.java` : Classe utilitaire pour le chiffrement/déchiffrement AES.

2.3 Flux de communication Nearby

1. **Advertising** : L'appareil se rend visible via Bluetooth.
 2. **Discovery** : L'appareil recherche les utilisateurs visibles autour de lui.
 3. **Connexion** : Lorsqu'un utilisateur est trouvé, on demande une connexion via `requestConnection`.
 4. **Lifecycle de la connexion** : Gestion des callbacks `onConnectionInitiated`, `onConnectionResult` et `onDisconnected`.
 5. **Échange de messages** : Les messages sont chiffrés avec AES avant envoi et déchiffrés à la réception.
-

3. Installation et configuration

3.1 Prérequis

- Android Studio (version récente)
- SDK Android avec Google Play Services
- Appareil Android compatible Bluetooth
- Permissions Android configurées dans `AndroidManifest.xml` :

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
```

3.2 Cloner et lancer le projet

```
git clone https://gitlab.lev-btssio.fr/owen.auriault/synchrodrops.git
cd synchrodrops
```

- Ouvrir le projet dans Android Studio.
 - Ajouter la clé AES dans `BuildConfig.AES_KEY`.
 - Lancer l'application sur un appareil physique (Nearby ne fonctionne pas sur émulateur pour Bluetooth).
-

4. Fonctionnalités détaillées

4.1 Advertising

startAdvertising() // Rends l'utilisateur visible

- Utilise `Nearby.getConnectionsClient(this).startAdvertising(...)`.
- Callback gère succès et échec.

4.2 Discovery

startDiscovery() // Recherche les utilisateurs autour

- Utilise `Nearby.getConnectionsClient(this).startDiscovery(...)`.
- Callback `EndpointDiscoveryCallback` permet d'ajouter les utilisateurs découverts dans `RecyclerView`.

4.3 Gestion des connexions

- `ConnectionLifecycleCallback` :
 - `onConnectionInitiated` : Affiche une boîte de dialogue pour accepter ou refuser la connexion.
 - `onConnectionResult` : Vérifie si la connexion est acceptée ou refusée.
 - `onDisconnected` : Nettoie la connexion si un utilisateur se déconnecte.

4.4 Envoi et réception de messages

- Messages chiffrés avec AES avant l'envoi :

```
String encryptedMessage = crypt.encrypt(message, key);
Payload bytesPayload = Payload.fromBytes(encryptedMessage.getBytes());
laConnexionNearby.sendPayload(endpointId, bytesPayload);
```

- Messages reçus sont déchiffrés dans `ClasseTraitementDonneeRecues` :

```
String decryptedMessage = crypt.decrypt(receivedString, key);
```

- Mise à jour `RecyclerView` pour afficher les messages.

5. UI / RecyclerView

- Liste des utilisateurs : `NearbyAdapter`
- Chat : `ChatAdapter`
- Messages différenciés par l'alignement et le fond selon l'expéditeur (`bg_message_me` ou `bg_message_other`).

6. Sécurité

- Chiffrement symétrique AES pour sécuriser les messages.
 - Clé stockée dans `BuildConfig.AES_KEY`.
 - Les utilisateurs doivent accepter la connexion pour commencer le chat.
-

7. Étapes pour recréer le projet rapidement

1. Créer un nouveau projet Android.
 2. Ajouter les permissions dans `AndroidManifest.xml`.
 3. Implémenter `MainActivity` avec Nearby (Advertising, Discovery, LifecycleCallback).
 4. Créer les modèles `UserNearby` et `MessageNearby`.
 5. Créer les adaptateurs `NearbyAdapter` et `ChatAdapter`.
 6. Ajouter la classe `AESUtils` pour le chiffrement/déchiffrement.
 7. Configurer la clé AES dans `BuildConfig`.
 8. Construire l'UI avec RecyclerView et boutons pour envoyer/chercher des utilisateurs.
 9. Tester sur deux appareils physiques pour valider la connexion et le chat.
-

8. Conseils & améliorations

- Ajouter une gestion des erreurs plus robuste sur Nearby.
- Ajouter un historique de messages persistants.
- Implémenter un chiffrement asymétrique pour plus de sécurité.
- Ajouter des notifications pour nouveaux messages reçus.